

HTTP/3-to-HTTP/1.1 Boundary Handling: A Source-Led Characterization of Six Open-Source Stacks

Turhan Acar
ttrnn47@gmail.com

2 October 2026

Abstract

HTTP/3 reverse proxies forwarding to HTTP/1.1 origins must reconcile stream completion, body-byte accounting, downstream framing, and backend connection reuse. We present a bounded source analysis using six illustrative cases: HAProxy, nginx, Envoy/QUICHE, Caddy/quic-go, nghttp3/ngtcp2 through nghttpx, and Apache Traffic Server. Publicly disclosed HAProxy and Envoy defects motivate an obligation-by-obligation evidence matrix, distinguishing supported local properties from partial and open paths. HAProxy contributes complementary remediation changes, not an independently traced end-to-end deployment; Envoy’s headers-only check is qualified by a default-on runtime guard. nginx and nghttpx expose different placements of length validation, but the inspected nginx reuse path remains unresolved. The ATS inspection supports only a no-early-credit observation at one adaptor, leaving end-to-end framing unresolved. No Caddy body-desynchronization result is claimed. Positive source conclusions identify a revision, path, and stated condition. The contribution is the bounded cross-implementation mapping and its explicit evidence gaps, not discovery of the public defects. No new vulnerability, dynamic experiment, or implementation-wide immunity is claimed.

Copyright 2026 Turhan Acar. Licensed under CC BY 4.0.

1 Introduction

Protocol translation preserves application semantics only if it also preserves message boundaries. HTTP/3 carries message bodies on framed QUIC streams, whereas HTTP/1.1 uses message-length rules on connections that may carry successive requests. A front end can parse individual inputs plausibly while still mishandling the transition between completion of a stream and completion of a message on a reusable backend connection. RFC 9114 defines HTTP/3 frame and message validity; RFC 9112 defines HTTP/1.1 framing and connection management [2, 1].

This paper asks a bounded question: *where, at specified source revisions, are delivered-byte accounting, message-completion validation, and backend reuse coupled, and how far does the inspected evidence support a conclusion?* The question is motivated by public defects, including HAProxy CVE-2026-90678 and CVE-2026-33555 and Envoy CVE-2026-48743 [3, 5, 6]. It does not ask whether any implementation is free of request smuggling in general.

The contribution is a cross-implementation mapping of four boundary obligations—accounting, completion, HTTP/1 framing, and reuse—at immutable revisions. The resulting matrix distinguishes a locally supported property from an untraced link rather than assigning one security rating to an

entire stack. This mapping is the present analysis; the motivating vulnerabilities and their fixes are public prior work, not new discoveries. The study is descriptive and source-led; it introduces neither an exploit nor a formal verification procedure. Two related engineering artifacts are documented separately in Appendix A and are not evidence for the central body-framing claims.

The six cases form an illustrative set drawn from the source investigations available to the author in September 2026, not a prespecified sampling universe. Each has publicly inspectable HTTP/3 frontend or integration code relevant to HTTP/1 forwarding; inclusion does not require a completed source argument. Caddy is retained to expose the distinction between a named candidate and an established result. No systematic search for all eligible proxies, exclusion census, or representativeness claim is made. Shared transport libraries do not make the stacks independent experimental samples: Envoy consumes QUICHE, Caddy consumes quic-go, and nghttpx consumes nghttp3/ngtcp2. Conclusions about an integration do not automatically transfer to other consumers of its libraries.

2 Scope and evidence semantics

2.1 Boundary under study

The inspected setting is an HTTP/3 front end translating requests to an HTTP/1.1 origin. The relevant failure family involves disagreement among body length, stream completion, and eligibility for subsequent use of an origin connection. Truncated DATA frames and a body-length declaration unmatched by delivered data are related but distinct cases: a DATA-parser guard alone does not establish coverage of a headers-only completion path. The HTTP/3 requirements for malformed messaging and truncated frames are separate inspection obligations [2].

The source inspection considers four obligations:

1. **Accounting (A)**: does the inspected body callback or adaptor credit delivered bytes rather than a declaration of future bytes?
2. **Completion (C)**: which paths reconcile frame completion and declared message length before treating a request as complete?
3. **HTTP/1 framing (F)**: does the inspected serializer derive emitted body framing from the bytes and completion state it is given?
4. **Reuse (R)**: what request/response state and error conditions govern returning an origin connection to a pool?

These questions are an analysis rubric, not a retrospectively claimed preregistration. A favorable answer at one layer may support a narrower conclusion than end-to-end containment. A correctness deviation also need not create a cross-request security effect.

2.2 Evidence states

The states in Table 1 describe *local evidence for each obligation*, not product security ratings:

Supported (S): named source statements support the local property under the stated conditions. This is not an exhaustive control-flow proof, a whole-obligation certificate, or coverage of all configurations.

Partial (P): a relevant guard or change has been identified, but the stated connection to the obligation is incomplete.

Open (O): the present evidence does not establish the requested disposition; this includes unin-

spected links.

Non-result: an attempted observation lacks a qualifying durable record or ended in an apparatus failure and is excluded from target conclusions.

An OPEN or NON-RESULT entry is not evidence that a target is vulnerable or immune. The table has no aggregate “complete” verdict: even a row of S entries remains limited to its named paths. Manual source analysis can identify guards and trace data flow; absence of an identified counterexample is not a proof that none exists. We therefore avoid implementation-wide “immune” or “unbypassable” labels.

3 Method and provenance

For each case, we identify a revision and the entrypoint already named in the source investigation or public remediation record. We then follow the relevant data callback or adaptor, its delivered-byte accounting, its completion predicate, and, where inspected, the HTTP/1 body writer and connection-detachment predicate. Each named link receives its own S, P, or O state; an untraced serializer or pool is not filled in by inference from a parser guard. The accompanying claim matrix records the inspected functions and the stopping boundary. This is a documentation protocol for the present readings, not an assertion that all alternative callbacks or configurations were searched. The dated source snapshot is the unit of analysis. Release names, version-bump commits, development snapshots, code fixes, backports, and merge commits are distinguished in Appendix B.

The detailed source readings originated in September 2026. On 2 October 2026, an AI-assisted author-side cross-check compared the recorded predicates with the cited pinned files and checked attribution and release statements against public change records. No independent human replication is claimed by this step. This paper does not assert that every current branch was audited. An immutable commit pins a past tree; it does not imply that the tree is the latest release, that it was built successfully, or that its behavior was measured.

The present result set is source-led. No new dynamic experiment was performed for this manuscript. Historical observations lacking a retained qualifying run record or ending in instrument error are non-results and are excluded, not used as comparative rows. Neither response counts nor timings from those records are used as target evidence. Public tests accompanying the engineering artifacts in Appendix A are upstream artifacts; we do not claim to have rerun them for this paper.

Source inspectability here means that a reader can locate the exact tree and inspect the named functions and conditions. It does not imply a systematic search over all source paths or a runnable dynamic-reproduction package. The paper includes no stimulus generator, attack payload, or exploit procedure.

4 Comparative source results

4.1 HAProxy: complementary remediation changes

The public CVE-2026-90678 record identifies affected stable ranges 3.3.0–3.3.14 and 3.4.0–3.4.4, and development versions 3.5-dev1–3.5-dev5; it explicitly marks 3.5-dev6 unaffected [3]. HAProxy’s stable-branch changelogs place the remediation in 3.3.15 and 3.4.5 [4]. These ranges do not mean that every deployment is exposed: the advisory concerns QUIC-enabled HTTP/3 front ends forwarding to HTTP/1.1 backends under its specified framing and connection-reuse conditions.

Case	Accounting	Completion	H1 framing	Reuse
HAProxy changes	O	S (fix)	O	P (fix)
nginx	S	S	S	O
Envoy/QUICHE	S	S	O	O
Caddy/quic-go	O	O	O	O
nghttpx stack	S	S	S	S
ATS	S	O	O	O

Table 1: Local source evidence, not security ratings. S: supported local property; P: partial; O: open. HAProxy records two complementary changes, not a single traced deployment. Envoy completion requires its named runtime guard. ATS accounting covers only its DATA adaptor. Detailed limits appear in the case subsections.

Two distinct changes support the remediation account. Commit [86a4ebc761a278838e8cb06f3a292282ba704c65](#) adds frame-completeness checks in `src/h3.c`, covering the FIN/EOM path and the DATA parsing loop [7]. Commit [4721a96f161b83cbb5015d15821e84e16fe21151](#) separately strengthens `src/mux_h1.c` to treat remaining short-message data as an error without requiring a Content-Length flag [8]. The earlier condition had covered Content-Length messages; this hardening extends the check to the chunked/no-Content-Length case.

The supported conclusion is that the remediation includes complementary H3 and H1 changes. This row does not trace one exact built tree through all four obligations and does not establish independence of the defenses. The H1 change is not characterized as an already-present independent defense of the affected 3.4.4 path. The analysis neither establishes exhaustive non-bypassability nor claims novelty in the public fix. The related CVE-2026-33555 is retained as public context for a distinct completion path, not collapsed into the truncated-DATA check [5].

4.2 nginx: delivered-byte accounting at a development snapshot

The inspected revision is [ef0aa967dce9d30b824d4c839d3579d2a17e0666](#). Its source version reads 1.31.7; it is treated as a mainline development snapshot, not as a released 1.31.7 package [9].

In `ngx_http_v3_request_body_filter`, the amount copied into the body chain is bounded by available buffer bytes and the remaining frame payload. The delivered-byte count increases by the copied amount. The end-of-input handling rejects a remaining frame payload and checks the delivered count against an existing Content-Length; absent such a declaration, the count is derived from received bytes [10]. The DATA parser keeps the declaration in parser state, while the standard HTTP proxy chunk writer sizes chunks from actual buffer contents and emits a terminal chunk only on its final-buffer marker [11, 12]. These predicates support the bounded source argument for body accounting and end validation.

The conclusion is scoped to the inspected body-filter path and the ordinary buffered reverse-proxy use of that path. It does not cover arbitrary modules, changed buffering settings, all upstream filters, or every error/reuse interleaving. No live rejection rate, comparative timing, or byte-capture result is claimed for this snapshot.

4.3 Envoy/QUICHE: a post-remediation, conditional disposition

The Envoy pin is `bfd41cd54d81cad5975b0443ab9c59543f1bbe18`, using QUICHE `5c9cc6b37f55a97071077a6190bf4f0bc7a09c1f`. The public advisory for CVE-2026-48743 records affected and fixed versions for a headers-only HTTP/3 request with an unsatisfied nonzero Content-Length on an HTTP/1 upstream path [6]. It lists fixes in 1.35.13, 1.36.9, 1.37.5, and 1.38.3. The inspected development revision is therefore analyzed as post-remediation, not as historically immune.

At this revision, `OnInitialHeadersComplete` applies a zero-byte end-of-stream accounting check before decoding completed headers when the relevant runtime guard is enabled. The shared accounting helper rejects an end-of-stream Content-Length mismatch. The body path separately accounts for delivered data [13, 14]. The headers-only repair is public commit `59c7458d4dc48705d4f7b74f8a92912b5ca79822` [15].

The runtime feature, `envoy.reloadable_features.quic_validate_headers_only_content_length`, is default-on at the inspected pin and can be disabled. The disposition depends on it remaining enabled [16]. This condition is part of the result, not a deployment footnote. QUICHE’s decoder interface distinguishes frame metadata from payload delivery; inspecting that decoder does not prove that every consumer handles those signals safely. The argument here is about Envoy’s inspected consumer path, including its validation, and makes no ecosystem-wide claim about QUICHE users.

4.4 Caddy/quic-go: body disposition remains open

The baseline Caddy source identity is version 2.11.4 at `e2eee6a7fce366321294c9c2a79f3146891dcdbdf`. The body-desynchronization question is OPEN in this paper. A narrative of an earlier paired experiment is insufficient to establish a reproducible measurement without its qualifying raw artifacts. No claim of wire equivalence, same-connection victim safety, or general body-framing containment is made.

Caddy remains in the matrix to make the coverage gap explicit. Its accepted upgraded-stream correctness change is supported by a separate public artifact and is discussed in Appendix A; that change does not answer the open body-framing question.

4.5 nghttp3/ngtcp2 through nghttpx: library and consumer obligations

The pinned integration comprises `nghttp3 2304973e5a0c8b1fa4bb380b47945a000357f87f`, `ngtcp2 3c23148ef32596cb75720b94615d9650cdec5391`, and `nghttp2/nghttpx 140157a8d751e9e8b0cc15bf9bc0e2ee0c1ed987`. These are source snapshots. The frontend analyzed is `nghttpx`, not Apache HTTP Server or a third-party `mod_http3` integration.

The inspected `nghttp3` DATA path delivers the available payload span to its application callback while keeping remaining frame length in parser state. The end-of-input state check rejects incomplete frames, and HTTP message completion reconciles received content with Content-Length before signaling clean completion [17, 18, 19]. In `nghttpx`, chunk emission uses the supplied data span; the request path and connection-detachment predicate connect completion to reuse eligibility [20, 21, 22].

The reuse predicate requires complete request and response states and drained request buffers, among its other conditions. Taken together, these inspected paths support the bounded argument for this integration. The result is not a successful build or a live test, and it does not establish that other consumers use the library correctly. The presence of `ngtcp2` in the provenance identifies the transport integration; it is not a claim that every `ngtcp2` transport path was independently audited.

4.6 Apache Traffic Server: a partial adaptor observation

At ATS commit `ca133303832de2cb0c1d5b2ea38c9d2cc5d53820`, an 11.0.0 development tree, `Http3StreamDataVIOAdaptor::handle_frame` transfers available data from the bounded frame reader, and `finalize` sets the sink byte count from completed delivery [23, 24]. This supports a local no-early-credit observation for the inspected DATA adaptor.

It does not establish the complete HTTP/1 origin framing, all forwarding modes, or the standalone-FIN/nonzero-Content-Length sibling. Those remain OPEN. ATS explicitly describes HTTP/3 and QUIC as experimental [25]; the presence of this code must not be confused with a mature default HTTP/3 deployment.

Two public HTTP/1 parser fixes provide adjacent context. CVE-2026-24033 is fixed in 9.2.15/10.1.4, while CVE-2025-65114 is fixed in 9.2.13/10.1.2 [26, 27]. Their branch-specific *code fixes* and release identities are distinguished in Appendix B. The lower 9.2.13/10.1.2 floor does not cover both issues. These HTTP/1 fixes do not resolve the open HTTP/3 questions in this paper.

5 Interpretation and limitations

The comparison separates four obligations that are often compressed into a single disposition: message-body accounting, completion validation, HTTP/1 serialization, and origin-connection lifecycle. A favorable local result does not fill another column. Coexistence of checks does not establish their independence, and public patch history is not equivalent to a full end-to-end source trace.

The observed diversity also limits simple labels. nginx validates through its body filter; Envoy needs a completion check outside the ordinary body-data path; nghttpx’s argument spans library validation and consumer reuse conditions. ATS shows why a favorable local adaptor observation cannot be promoted to an end-to-end conclusion. Caddy shows why a retained narrative and an accepted but different correctness fix cannot fill an empirical gap.

Internal validity. Source readings may miss alternate callbacks, build options, error paths, runtime overrides, or transformations outside the inspected boundary. No model checker or exhaustive control-flow analysis was used. An explicit predicate at a pinned location supports a narrow local inference; an instrument-failing run supports no target-behavior inference. Neither establishes universal behavior. AI-assisted cross-checking may repeat a shared error and is not independent replication.

External validity. The sample is purposive, versions are heterogeneous, and some pins are development snapshots. No prevalence, deployment coverage, performance, or security ranking is inferred. The results do not transfer automatically to newer releases or other consumers of the same library.

Evidence completeness. No dynamic result is claimed. Historical records excluded as non-results are not used to raise confidence in a source conclusion. The accepted upstream patches document specific fixes and public review history; their tests are not presented as newly executed experiments. Reproducibility is limited to source inspection using the supplied identities and anchors.

Scope. The article covers a narrow body-completion family and two adjacent correctness contributions. It does not characterize all HTTP/1 or HTTP/2 parser discrepancies, authentication, cache behavior, or closed-source edges. An unresolved path is an evidence limit rather than a vulnerability allegation.

6 Related work

Linhart et al. described HTTP request smuggling through inconsistent message interpretation in a 2005 Watchfire white paper [35]. Kettle’s practitioner research revisited connection desynchronization and demonstrated the additional risks created by HTTP/2 translation [36, 37]. These works establish the context for inspecting boundaries between parsers and protocol versions; they do not support a blanket conclusion about the pins in this study.

T-Reqs applies grammar-based differential fuzzing to HTTP request smuggling [38]. The HTTP Garden studies origin interpretations and gateway transformations using differential fuzzing of request streams [39]. Those methods systematically seek discrepancies and measure outcomes. This paper instead contributes a bounded comparative source analysis and an explicit record of where the inspected evidence stops. It neither replaces empirical differential testing nor infers coverage of a larger input space from a small set of paths.

FRAMESHIFTER directly studies HTTP/2-to-HTTP/1 conversion anomalies through grammar-based dynamic fuzzing [40]. HDiff combines specification-derived rules with differential testing, while Gudifu uses coverage-guided differential fuzzing for HTTP request parsing [41, 42]. The present analysis differs in protocol focus and evidence type: it maps named H3-to-H1 source obligations and untraced links at fixed revisions, without generating a test corpus or claiming dynamic discovery coverage. This methodological distinction is not a claim of broader coverage or superiority.

The protocol standards supply the expected message and frame semantics [1, 2]. The public advisories and individual commits supply historical defect and remediation evidence. The two correctness contributions are identified through their upstream records, with attribution and severity preserved. The absence of a newly disclosed vulnerability is compatible with a source-analysis contribution; the value of the present work rests on the specificity and inspectability of its arguments.

7 Ethics, assistance, and artifact availability

This manuscript is based on public source, public advisories, and accepted upstream changes. It reports no new vulnerability, includes no operational attack instructions, and makes no claim about a third party’s deployment. No live target or local proof was executed for this manuscript preparation. The HAProxy and Caddy correctness changes were already submitted and accepted upstream.

Anthropic Claude and OpenAI Codex substantially assisted source navigation, cross-checking, drafting, and document preparation. Turhan Acar directed the work and is responsible for its claims, citations, and final publication decision. Automated author-side checks do not constitute independent peer review. No AI system is an author.

The canonical manuscript is the self-contained `main.tex` file, which includes its bibliography. Public source references use full commit identities. The accompanying `source-pins.md` and `claim-matrix.md` restate the inventory and claim boundaries for inspection. There is no separate dynamic dataset or reproduction harness supporting this paper, and no external artifact URL is required to resolve its source references.

8 Conclusion

At the inspected pins, source arguments about body accounting and completion differ in both location and scope. HAProxy supplies complementary remediation changes; Envoy’s completion check depends on a runtime condition and does not by itself close the H1 path. nginx supports local accounting, completion, and framing arguments with reuse untraced; nghttpx has named evidence at all four obligations under its stated scope. ATS remains partial, and the Caddy body question remains open. The obligation-by-obligation matrix exposes these differences without converting missing evidence into a negative result or a product-wide assurance.

A Related engineering artifacts

These case studies concern adjacent validation and lifecycle boundaries. They provide publicly inspectable engineering artifacts and are kept separate from the H3 body-disposition matrix. Acceptance of a patch is evidence that it was accepted; it is not an independent security proof for a whole product.

A.1 HAProxy HTTP/2 response pseudo-header validation

Turhan Acar authored commit `4ddf219bb399334e00a0b9af2b155884befd4b04`, committed by Willy Tarreau on 24 September 2026 [28]. The change modifies only `src/h2.c`, adding `H2_PHDR_FND_PROT` to the response-side rejection mask for request-only pseudo-headers in `h2_make_htx_response`. The upstream classification is `BUG/MINOR`. The protocol context is the restriction of request pseudo-headers from responses and the Extended CONNECT `:protocol` field [30, 31]; the patch cites the historical RFC 7540 [29].

The accepted commit states “no known security impact.” Its assessment is that the offending pseudo-header is absent from emitted HTTP/1 and that the effect is tolerating an invalid response rather than rejecting it. We retain that bounded assessment and claim no stronger security conclusion. This contribution is unrelated to the H3 remediation commits discussed above.

A.2 Caddy upgraded-stream half-close propagation

Caddy PR #8027, authored by Turhan Acar, was squash-merged on 16 September 2026 as `9be8fb279444ec633ae2eea4590a180e1143c8f6` [32, 33]. This later master change is not present in the v2.11.4 baseline. It changes the upgraded-stream copier in `streaming.go` and accompanying tests in `streaming_test.go`. A clean directional EOF is propagated to a destination supporting `CloseWrite`, allowing the other direction to finish; errors, cancellation, and timeout retain teardown behavior. The public change includes tests covering this lifecycle distinction.

The linked issue describes a correctness/interop defect and explicitly makes no security claim [34]. The public work was demonstrated on an HTTP/1.1 Upgrade path, so it is not characterized here as an HTTP/3-specific result. The accepted change provides no evidence of cross-user containment, response-queue poisoning, or global security absence. The immutable squash merge is the contribution pin; a mutable milestone is not treated as a shipped release. The PR’s final branch head, `2ec7211b42e2bff1c32741b6e271a92bc2da690a`, is a merge of the base branch and must not be mistaken for the author’s feature commit.

B Immutable source inventory

All values below are full Git commit identities. Source-only entries identify inspected trees, not tested binaries. Links in the bibliography identify the relevant files/functions at these pins.

Component / purpose	Full identity and qualification
HAProxy H3 completeness	86a4ebc761a278838e8cb06f3a292282ba704c65; code fix in <code>src/h3.c</code> .
HAProxy H1 reuse hardening	4721a96f161b83cbb5015d15821e84e16fe21151; separate code fix in <code>src/mux_h1.c</code> .
nginx source snapshot	ef0aa967dce9d30b824d4c839d3579d2a17e0666; source version 1.31.7, development snapshot.
Envoy source snapshot	bfd41cd54d81cad5975b0443ab9c59543f1bbe18; 1.40.0-dev.
QUICHE dependency	5c9cc6b37f55a97071077a6190bf4f0bc7a09c1f; Envoy dependency; conclusions remain consumer-specific.
Envoy headers-only repair	59c7458d4dc48705d4f7b74f8a92912b5ca79822; runtime-guarded code fix.
Caddy baseline	e2eee6a7fce366321294c9c2a79f3146891dcbdf; v2.11.4, body question open.
nghttp3	2304973e5a0c8b1fa4bb380b47945a000357f87f; development snapshot.
ngtcp2	3c23148ef32596cb75720b94615d9650cdec5391; transport provenance.
nghttp2 / nghttpx	140157a8d751e9e8b0cc15bf9bc0e2ee0c1ed987; frontend source snapshot.
ATS H3 inspection	ca133303832de2cb0c1d5b2ea38c9d2cc5d53820; development tree, experimental H3.
ATS CVE-2026-24033 code fixes	10.1.x: bb7f7263dff737c2c895dbb225b5415f93b804d1. 9.2.x backport: e44213f8ec0d1df65de99a15fe2f59c16e01f30b.
ATS CVE-2025-65114 code fixes	10.1.x: 7c2c689e5843a3be70b775600b125a2957422209. 9.2.x backport: e5accd7929c5cb96a01cc9afda1f6336dab59b64.
ATS release identities	10.1.4: f8be6725daee427fbf88f166f14b10cb5525b220; 9.2.15: f671e525110be50452f607fc1f2657f3dba7fe3b. 10.1.2: fd932cb399f0b18d1044700d34bc028edabcb82d; 9.2.13: 2c83b07b60ec73e4aa48abc10e82b248e3f2a23b. These are not the code fixes above.
HAProxy correctness contribution	4ddf219bb399334e00a0b9af2b155884befd4b04; response validation, BUG/MINOR.
Caddy correctness contribution	9be8fb279444ec633ae2eea4590a180e1143c8f6; accepted squash merge for PR #8027.

References

- [1] R. Fielding, M. Nottingham, and J. Reschke, eds. *HTTP/1.1*. RFC 9112, June 2022. <https://www.rfc-editor.org/rfc/rfc9112.html>.
- [2] M. Bishop, ed. *HTTP/3*. RFC 9114, June 2022. Sections 4.1.2 and 7.1. <https://www.rfc-editor.org/rfc/rfc9114.html>.
- [3] CVE Program. *CVE-2026-90678*. Public vulnerability record, 2026. <https://www.cve.org/CVERecord?id=CVE-2026-90678>.
- [4] HAProxy project. Stable-branch changelogs, entries for 3.3.15 and 3.4.5, 2026. <https://www.haproxy.org/download/3.3/src/CHANGELOG> and <https://www.haproxy.org/download/3.4/src/CHANGELOG>.

- [5] CVE Program. *CVE-2026-33555*. Public vulnerability record, 2026. <https://www.cve.org/CVERecord?id=CVE-2026-33555>.
- [6] Envoy project. *HTTP/3 to HTTP/1 request smuggling via headers-only request with nonzero Content-Length*. GHSA-8phg-2h2q-jgxf / CVE-2026-48743, 23 June 2026. <https://github.com/envoyproxy/envoy/security/advisories/GHSA-8phg-2h2q-jgxf>.
- [7] HAProxy project. *BUG/MAJOR: h3: reject H3 truncated frames*. Commit, 2026. <https://github.com/haproxy/haproxy/commit/86a4ebc761a278838e8cb06f3a292282ba704c65>.
- [8] HAProxy project. *MEDIUM: mux-h1: Harden test for short data at end of stream*. Commit, 2026. <https://github.com/haproxy/haproxy/commit/4721a96f161b83cbb5015d15821e84e16fe21151>.
- [9] nginx project. *Source version 1.31.7*. Version-bump commit, 2026. <https://github.com/nginx/nginx/commit/ef0aa967dce9d30b824d4c839d3579d2a17e0666>.
- [10] nginx project. *ngx_http_v3_request_body_filter*, pinned source, lines 1660–1749. https://github.com/nginx/nginx/blob/ef0aa967dce9d30b824d4c839d3579d2a17e0666/src/http/v3/ngx_http_v3_request.c#L1660-L1749.
- [11] nginx project. *ngx_http_v3_parse_data*, pinned source. https://github.com/nginx/nginx/blob/ef0aa967dce9d30b824d4c839d3579d2a17e0666/src/http/v3/ngx_http_v3_parse.c#L1800-L1869.
- [12] nginx project. *Standard HTTP proxy body-output filter*, pinned source. https://github.com/nginx/nginx/blob/ef0aa967dce9d30b824d4c839d3579d2a17e0666/src/http/modules/ngx_http_proxy_module.c#L1635-L1760.
- [13] Envoy project. *EnvoyQuicServerStream::OnInitialHeadersComplete*, pinned source, lines 184–250. https://github.com/envoyproxy/envoy/blob/bfd41cd54d81cad5975b0443ab9c59543f1bbe18/source/common/quic/envoy_quic_server_stream.cc#L184-L250. Body accounting: https://github.com/envoyproxy/envoy/blob/bfd41cd54d81cad5975b0443ab9c59543f1bbe18/source/common/quic/envoy_quic_server_stream.cc#L263-L313.
- [14] Envoy project. *updateReceivedContentBytes*, pinned source, lines 207–219. https://github.com/envoyproxy/envoy/blob/bfd41cd54d81cad5975b0443ab9c59543f1bbe18/source/common/quic/envoy_quic_stream.h#L207-L219.
- [15] Envoy project. *quic: validate HTTP/3 headers-only request and response content-length*. Commit, 2026. <https://github.com/envoyproxy/envoy/commit/59c7458d4dc48705d4f7b74f8a92912b5ca79822>.
- [16] Envoy project. *Runtime feature defaults and registration*, pinned source. https://github.com/envoyproxy/envoy/blob/bfd41cd54d81cad5975b0443ab9c59543f1bbe18/source/common/runtime/runtime_features.cc#L20-L32; guard registration: https://github.com/envoyproxy/envoy/blob/bfd41cd54d81cad5975b0443ab9c59543f1bbe18/source/common/runtime/runtime_features.cc#L147.
- [17] nghttp3 project. *nghttp3_conn_read bidi DATA and end-of-input states*, pinned source, lines 1667–1835. https://github.com/nghttp2/nghttp3/blob/2304973e5a0c8b1fa4bb380b47945a000357f87f/lib/nghttp3_conn.c#L1667-L1835.
- [18] nghttp3 project. *nghttp3_http_on_remote_end_stream*, pinned source, lines 629–648. https://github.com/nghttp2/nghttp3/blob/2304973e5a0c8b1fa4bb380b47945a000357f87f/lib/nghttp3_http.c#L629-L648.
- [19] nghttp3 project. *Request MSG_END processing*, pinned source. https://github.com/nghttp2/nghttp3/blob/2304973e5a0c8b1fa4bb380b47945a000357f87f/lib/nghttp3_stream.c#L1057-L1139.
- [20] nghttp2 project. *nghttpx push_upload_data_chunk and end_upload_data*, pinned source. https://github.com/nghttp2/nghttp2/blob/140157a8d751e9e8b0cc15bf9bc0e2ee0c1ed987/src/shrpx_http_downstream_connection.cc#L725-L815.

- [21] nghttp2 project. `nghttpx Downstream::can_detach_downstream_connection`, pinned source. https://github.com/nghttp2/nghttp2/blob/140157a8d751e9e8b0cc15bf9bc0e2ee0c1ed987/src/shrpx_downstream.cc#L1150-L1157.
- [22] nghttp2 project. `nghttpx http_recv_data` and `http_stream_close`, pinned source. https://github.com/nghttp2/nghttp2/blob/140157a8d751e9e8b0cc15bf9bc0e2ee0c1ed987/src/shrpx_http3_upstream.cc#L2424-L2575.
- [23] Apache Traffic Server project. `Http3StreamDataVIOAdaptor::handle_frame` and `finalize`, pinned source. <https://github.com/apache/trafficserver/blob/ca133303832de2cb0c1d5b2ea38c9d2cc5d53820/src/proxy/http3/Http3StreamDataVIOAdaptor.cc#L45-L77>.
- [24] Apache Traffic Server project. DATA readiness and bounded frame dispatch, pinned source. <https://github.com/apache/trafficserver/blob/ca133303832de2cb0c1d5b2ea38c9d2cc5d53820/src/proxy/http3/Http3Frame.cc#L248-L258> and <https://github.com/apache/trafficserver/blob/ca133303832de2cb0c1d5b2ea38c9d2cc5d53820/src/proxy/http3/Http3FrameDispatcher.cc#L102-L145>.
- [25] Apache Traffic Server project. *Security policy*, including experimental HTTP/3 and QUIC status. Accessed 2 October 2026. <https://github.com/apache/trafficserver/security>.
- [26] Apache Traffic Server project. *July 2026 CVE release*, CVE-2026-24033. <https://trafficserver.apache.org/security-2026-07.html>.
- [27] Apache Software Foundation. *CVE-2025-65114: malformed chunked message body allows request smuggling*. Vendor advisory, 2026. <https://lists.apache.org/thread/2s11roxlv1j8ph6q52rqo1klv101n14q>. Record: <https://nvd.nist.gov/vuln/detail/CVE-2025-65114>.
- [28] T. Acar. *BUG/MINOR: h2: reject :protocol pseudo-header in H2 responses*. HAProxy commit accepted by W. Tarreau, 24 September 2026. <https://github.com/haproxy/haproxy/commit/4ddf219bb399334e00a0b9af2b155884befd4b04>.
- [29] M. Belshe, R. Peon, and M. Thomson, eds. *Hypertext Transfer Protocol Version 2 (HTTP/2)*. RFC 7540, May 2015, Section 8.1.2.1. Historical specification cited by the patch; superseded by RFC 9113. <https://www.rfc-editor.org/rfc/rfc7540.html#section-8.1.2.1>.
- [30] M. Thomson and C. Benfield, eds. *HTTP/2*. RFC 9113, June 2022, Section 8.3. <https://www.rfc-editor.org/rfc/rfc9113.html#section-8.3>.
- [31] P. McManus. *Bootstrapping WebSockets with HTTP/2*. RFC 8441, September 2018. <https://www.rfc-editor.org/rfc/rfc8441.html>.
- [32] T. Acar. *reverseproxy: propagate TCP half-close on upgraded streams*. Caddy pull request 8027, merged 16 September 2026. <https://github.com/caddyserver/caddy/pull/8027>.
- [33] Caddy project. Accepted squash merge for PR 8027, authored by T. Acar. <https://github.com/caddyserver/caddy/commit/9be8fb279444ec633ae2eea4590a180e1143c8f6>.
- [34] T. Acar. *reverse_proxy: client half-close aborts the backend connection on upgraded streams*. Caddy issue 8026, 2026. <https://github.com/caddyserver/caddy/issues/8026>.
- [35] C. Linhart, A. Klein, R. Heled, and S. Orrin. *HTTP Request Smuggling*. Watchfire white paper, 2005. Original document preserved at <https://www.cgisecurity.com/lib/HTTP-Request-Smuggling.pdf>.
- [36] J. Kettle. *HTTP Desync Attacks: Request Smuggling Reborn*. PortSwigger Research, 7 August 2019. <https://portswigger.net/research/http-desync-attacks-request-smuggling-reborn>.
- [37] J. Kettle. *HTTP/2: The Sequel is Always Worse*. PortSwigger Research, 5 August 2021. <https://portswigger.net/research/http2>.

- [38] B. Jabiyeve, S. Sprecher, K. Onarlioglu, and E. Kirda. T-Reqs: HTTP Request Smuggling with Differential Fuzzing. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1805–1820, 2021. <https://doi.org/10.1145/3460120.3485384>.
- [39] B. Kallus, P. Anantharaman, M. E. Locasto, and S. W. Smith. *The HTTP Garden: Discovering Parsing Vulnerabilities in HTTP/1.1 Implementations by Differential Fuzzing of Request Streams*. arXiv:2405.17737, 2024. <https://doi.org/10.48550/arXiv.2405.17737>.
- [40] B. Jabiyeve, S. Sprecher, A. Gavazzi, T. Innocenti, K. Onarlioglu, and E. Kirda. FRAMESHIFTER: Security Implications of HTTP/2-to-HTTP/1 Conversion Anomalies. In *31st USENIX Security Symposium*, pp. 1061–1075, 2022. <https://www.usenix.org/conference/usenixsecurity22/presentation/jabiyeve>.
- [41] K. Shen, J. Lu, Y. Yang, J. Chen, M. Zhang, H. Duan, J. Zhang, and X. Zheng. HDiff: A Semi-automatic Framework for Discovering Semantic Gap Attack in HTTP Implementations. In *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pp. 1–13, 2022. <https://doi.org/10.1109/DSN53405.2022.00014>.
- [42] B. Jabiyeve, A. Gavazzi, K. Onarlioglu, and E. Kirda. Gudifu: Guided Differential Fuzzing for HTTP Request Parsing Discrepancies. In *27th International Symposium on Research in Attacks, Intrusions and Defenses*, pp. 235–247, 2024. <https://doi.org/10.1145/3678890.3678904>.